① Let $L_1 = \{a^m b^n c^n \mid m, n \geq 0\}$

and $L_2 = \{a^m b^m c^n \mid m, n \geq 0\}$

L_1 & L_2 are context free languages as we can generate CFGs for them.

Now consider $L = L_1 \cap L_2$

$L_1 \cap L_2 = \{a^n b^n c^n \mid n \geq 0\}$

L is not a context free language (prove using pumping lemma).

Hence proved that context free languages are not closed under intersection operation.

Let's suppose context free langs are not closed under complementation.

$$L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$$

where $\overline{L_1}$ and $\overline{L_2}$ are content-free languages with the above suppositon.

$\overline{L_1} \cup \overline{L_2}$ is also a content free lang as ~~the~~ CFL are closed under union.

$\overline{\overline{L_1} \cup \overline{L_2}}$ must also be CFL then.

However, $L_1 \cap L_2$ is not a CFL.

Therefore our above suppositon is wrong that CFLs are closed under complementation (proof by contradiction).

(2) Let L be an infinite Turing recognizable language. If L is Turing recognizable, then there must be an enumerator for it.
Let the enumerator be E.

A enumerator E' exists that runs E and prints only those strings printed by E that are longer than previous strings printed by E'.

E' prints strings that belong to L.
E' is an enumerator for L.

A Turing Machine D exists that runs E'. If any string printed by E' matches the input string, then accept.
If any string printed by E' is longer than input string, then reject.

Since all shorter strings have been already printed, this implies that input string does not belong to D(L).

D is a decider and it only accepts strings that belong to L. Therefore D(L) is a decidable subset for L.

Hence, every infinite Turing recognizable language has a Turing decidable subset.

(3) $ALL_{DFA} = \{ \langle A \rangle \mid A \text{ is a DFA and } L(A) = \Sigma^* \}$

Turing Machine for ~~ALLDFA~~:

→ an input $\langle A \rangle$

(1) check if $\langle A \rangle$ is encoding of a valid DFA. if not, reject.

(2) If the start state is not an accepting state, reject. else mark it.

(3) Until no new states are marked
→ mark all those states that can be reached from already marked states by single transition AND are accepting states.

(4) if no states are left, accept. else, reject.

(4) $T = \{a^i b^j c^k \mid i, j, k \in N\}$

Create an infinite 2D matrix to store all strings in B where

$B = \{a^i b^j \mid i, j \in N\}$

Each row in matrix corresponds to i (ie number of a).

Each column in matrix corresponds to j (i.e number of b)

	0	1	2
0	ε	b	bb
1	a	ab	abb
2	aa	- - -	
	⋮	⋮	⋮

⋮

using diagonalization it can be proved that this set is countable. This implies that it has same no. of elements as $\mathbb{N}$.

Create another infinite 2D matrix to store all strings in T.

→ Each row corresponds to unique element of B.

→ Each col represents to k (i.e number of C)

→ Again, use diagonalization to prove set T is countable

(5) There are two TMs R_1 and R_2 that recognize L_1 and L_2.

Suppose $L = L_1 L_2$ (concatenation)

constructing a TM for L

TM will be multitape TM (has three tapes)

1) first tape will have the original input.

2) tape 2 will be divided into blocks separated by a delimiter eg # (given that # $\notin \Sigma$).

Each block will contain the input string divided into two substrings

Division can be represented by another special symbol not in Σ.

There will be n such blocks where n is the length of input string.

(3) The last tape will store state of each block.

TM:

For each block

(1) Run 1 step of R_1 on 1st substring

(2) Run 1 step of R_2 on 2nd substring

(3) Write states for both step 1 & 2 on tape 3 in the correct block.

if both R_1 and R_2 reach accepting state for any block, accept.

if either R_1 or R_2 goes into reject state for all blocks, reject.